

RX 인터럽트 기반 큐 전환 절차

- 작성일: 2026-04-15
- 관련: [LoRa_RX 문제와 목표 아키텍처.pdf](#), [lora_module.c](#) 서비스 루프

1. 배경

현재 `lora_service_thread`는 `lora_recv(500ms)`를 반복 호출하는 폴링 구조이다. 목표는 ****스레드가 큐만 대기 (K_FOREVER)****하고, RX는 **인터럽트 → 큐 적재**로 전달하는 구조이다.

Zephyr는 `lora_recv_async()` 비동기 수신 API를 제공한다.

```
// zephyr/include/zephyr/drivers/lora.h
typedef void (*lora_recv_cb)(const struct device *dev,
                             uint8_t *data, uint16_t size,
                             int16_t rssi, int8_t snr,
                             void *user_data);

int lora_recv_async(const struct device *dev, lora_recv_cb cb, void *user_data);
```

- 콜백은 **ISR 컨텍스트**에서 호출됨
- `cb = NULL` 전달 시 진행 중인 비동기 수신 취소

2. 설계 진화 — 단일 큐에서 우선순위 분리까지

2.1 1차 설계: 통합 단일 큐

가장 단순한 접근으로, 모든 일감을 하나의 큐에 넣는다.

```
[SX1262 DI01 인터럽트]
    |
    ▼
[lor_recv_async 콜백] ← ISR 컨텍스트
    |
    | k_msgq_put(&lora_work_queue, &rx_msg)
    ▼
[lor_svc 스레드] ← k_msgq_get(K_FOREVER)로 대기
    |
    ├── RX_PACKET 메시지 → lora_handle_rx_buf()
    ├── TX_REQUEST 메시지 → lora_do_tx()
    └── ACK_TIMEOUT 메시지 → lora_handle_ack_timeout()
```

2.2 단일 큐의 문제: RX 폭주 시 TX 기아(starvation)

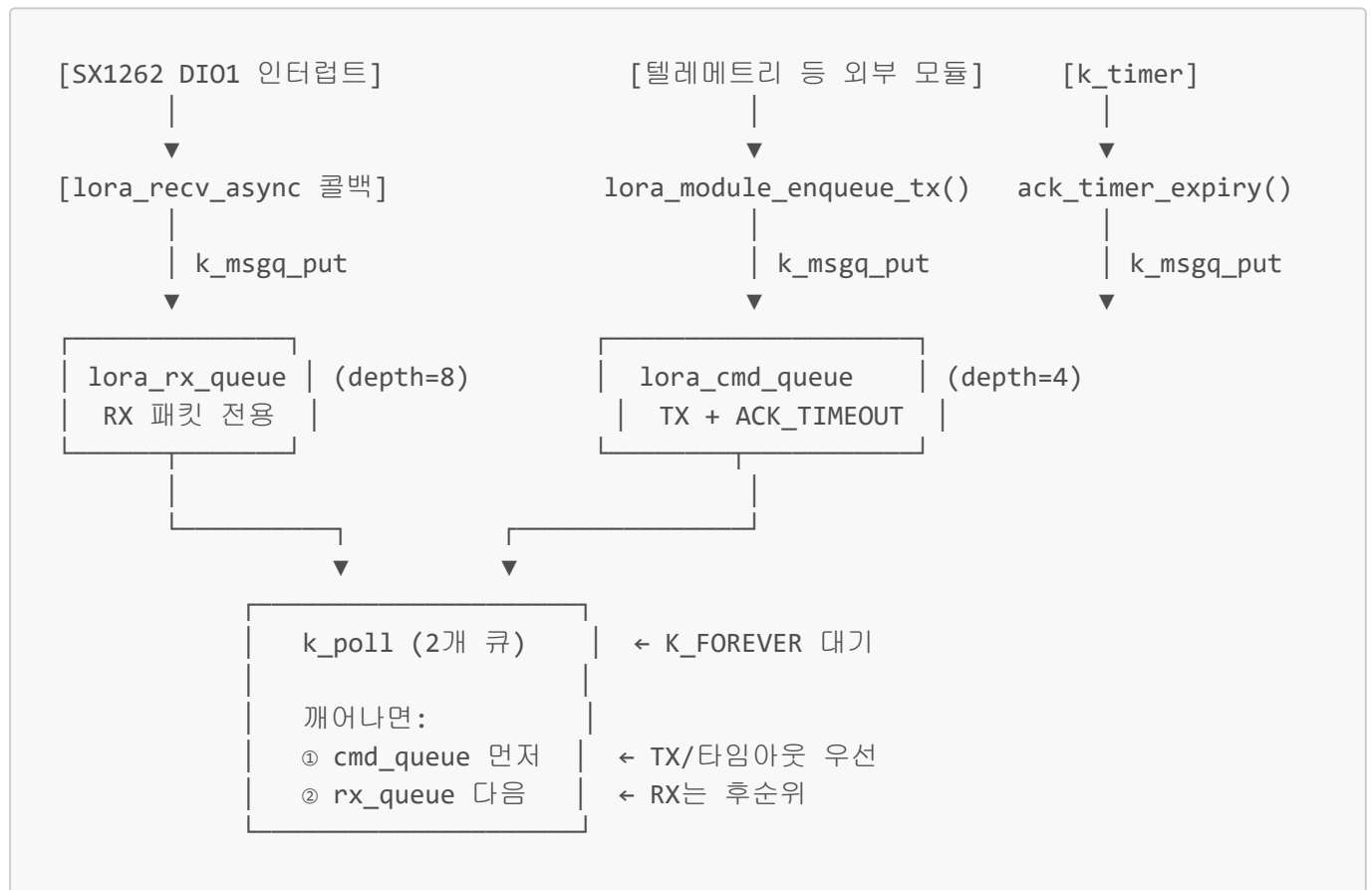
단일 큐는 **FIFO**이므로, RX가 계속 들어오면 심각한 문제가 발생한다.

큐 상태: [RX] [RX] [RX] [RX] [TX] [RX] [RX] [RX] ...
 ↑
 여기까지 밀림

문제	설명
TX 지연	TX_REQUEST가 큐에 들어가도, 앞에 쌓인 RX를 다 처리한 후에야 실행
큐 포화	큐(depth=8)가 RX로 가득 차면 TX_REQUEST 큐잉 자체가 실패
ACK 타임아웃 지연	ACK_TIMEOUT 메시지도 RX 뒤에 밀려 정확한 타이밍에 처리 불가

2.3 최종 설계: 우선순위 분리 (2큐 + k_poll)

큐를 **RX 전용**과 ****명령 전용(TX/타임아웃)****으로 분리하고, **k_poll**로 양쪽을 동시 대기하되 명령 큐를 우선 처리한다.



3. 최종 설계 — 구현 절차

3.1 메시지 타입 정의

```
/* — RX 큐용 — */
struct lora_rx_msg {
    uint8_t data[16];
```

```

    int16_t rssi;
    int8_t snr;
};

/* — 명령 큐용 — */
typedef enum {
    LORA_CMD_TX_REQUEST,
    LORA_CMD_ACK_TIMEOUT,
} lora_cmd_type_t;

struct lora_cmd_msg {
    lora_cmd_type_t type;
    union {
        struct {
            uint8_t data[16];
            uint8_t len;
            bool need_ack;
        } tx;
    };
};

/* — 큐 정의 — */
K_MSGQ_DEFINE(lora_rx_queue, sizeof(struct lora_rx_msg), 8, 4); // RX 전용
K_MSGQ_DEFINE(lora_cmd_queue, sizeof(struct lora_cmd_msg), 4, 4); // TX + 타임아웃

```

3.2 비동기 RX 콜백 — RX 큐에 적재

ISR 컨텍스트에서 호출되므로 최소한의 처리만 수행한다.

```

static void lora_rx_async_cb(const struct device *dev,
                             uint8_t *data, uint16_t size,
                             int16_t rssi, int8_t snr,
                             void *user_data)
{
    struct lora_rx_msg msg;

    if (size <= 16) {
        memcpy(msg.data, data, size);
    }
    msg.rssi = rssi;
    msg.snr = snr;

    k_msgq_put(&lora_rx_queue, &msg, K_NO_WAIT);
}

```

3.3 ACK 타임아웃 — 명령 큐에 적재

시간 기반 이벤트도 큐를 거쳐 워커가 처리한다.

```

static struct k_timer ack_timer;

static void ack_timer_expiry(struct k_timer *t)
{
    struct lora_cmd_msg msg = { .type = LORA_CMD_ACK_TIMEOUT };
    k_msgq_put(&lora_cmd_queue, &msg, K_NO_WAIT);
}

// 초기화 시
k_timer_init(&ack_timer, ack_timer_expiry, NULL);

// ACK 대기 시작 시
k_timer_start(&ack_timer, K_MSEC(ACK_TIMEOUT_MS), K_NO_WAIT);

// ACK 수신 시 타이머 취소
k_timer_stop(&ack_timer);

```

3.4 TX 요청 — 명령 큐에 적재

외부 모듈은 명령 큐에 TX_REQUEST를 넣는다.

```

bool lora_module_enqueue_tx(LoraPayloadType type,
                           const uint8_t *payload, uint8_t len,
                           bool need_ack)
{
    struct lora_cmd_msg msg = { .type = LORA_CMD_TX_REQUEST };

    if (payload == NULL || len != LORA_FRAME_BYTES) {
        return false;
    }

    memcpy(msg.tx.data, payload, len);
    msg.tx.len = len;
    msg.tx.need_ack = need_ack;

    return k_msgq_put(&lora_cmd_queue, &msg, K_NO_WAIT) == 0;
}

```

3.5 서비스 스레드 — k_poll 2큐 동시 대기 + 명령 우선

```

static void lora_service_thread(void *p1, void *p2, void *p3)
{
    /* ① 비동기 RX 시작 - 이후 수신은 콜백이 rx_queue에 넣음 */
    lora_recv_async(lora_dev, lora_rx_async_cb, NULL);

    for (;;) {
        struct lora_cmd_msg cmd;
        struct lora_rx_msg rx;

```

```

/* ㉔ 명령 큐 우선 확인 (TX / ACK 타임아웃) */
if (k_msgq_get(&lora_cmd_queue, &cmd, K_NO_WAIT) == 0) {
    switch (cmd.type) {
        case LORA_CMD_TX_REQUEST:
            lora_rcv_async(lora_dev, NULL, NULL); /* RX 중지 */
            lora_do_tx(&cmd.tx);
            lora_rcv_async(lora_dev, lora_rx_async_cb, NULL); /* RX 재개 */
            break;
        case LORA_CMD_ACK_TIMEOUT:
            lora_handle_ack_timeout_locked();
            break;
    }
    continue; /* 명령 처리 후 다시 명령 큐부터 확인 */
}

/* ㉕ RX 큐 확인 */
if (k_msgq_get(&lora_rx_queue, &rx, K_NO_WAIT) == 0) {
    lora_handle_rx_buf(rx.data, 16);
    continue;
}

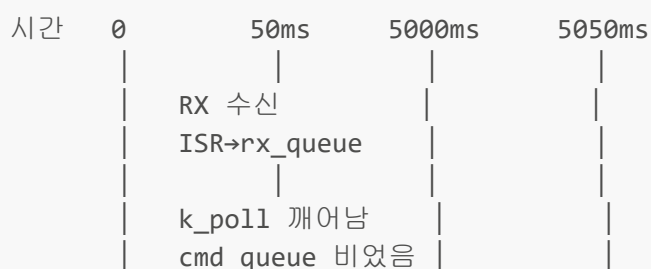
/* ㉖ 둘 다 비었으면 - k_poll로 양쪽 큐 동시 대기 (CPU sleep 가능) */
struct k_poll_event events[2] = {
    K_POLL_EVENT_INITIALIZER(
        K_POLL_TYPE_MSGQ_DATA_AVAILABLE,
        K_POLL_MODE_NOTIFY_ONLY,
        &lora_cmd_queue),
    K_POLL_EVENT_INITIALIZER(
        K_POLL_TYPE_MSGQ_DATA_AVAILABLE,
        K_POLL_MODE_NOTIFY_ONLY,
        &lora_rx_queue),
};

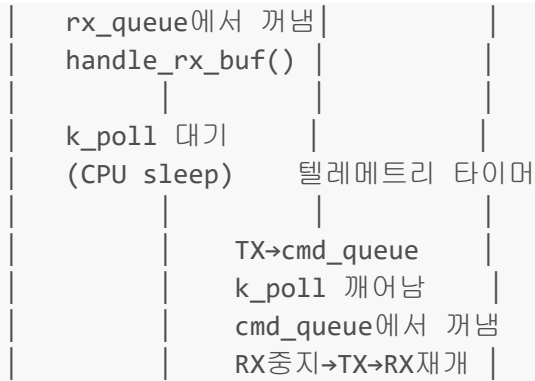
k_poll(events, 2, K_FOREVER);
/* 깨어나면 루프 처음으로 → cmd_queue부터 다시 확인 */
}
}

```

4. 동작 시나리오

4.1 정상 동작 (RX 간헐적)





4.2 RX 폭주 시 (TX 보호)

큐 상태:

cmd_queue: [TX_REQUEST]

rx_queue: [RX] [RX] [RX] [RX] [RX] [RX]

스레드 처리 순서:

- ① cmd_queue 확인 → TX_REQUEST 발견 → TX 즉시 실행!
- ② cmd_queue 확인 → 비었음
- ③ rx_queue 확인 → RX 처리
- ④ cmd_queue 확인 → 비었음 (매번 우선 확인)
- ⑤ rx_queue 확인 → RX 처리
- ...

- RX가 아무리 쌓여도 **TX는 다음 루프에서 즉시 실행**
- cmd_queue가 별도이므로 **RX 큐 포화가 TX에 영향 없음**

4.3 유힬 시 (저전력)

cmd_queue: (비었음)

rx_queue: (비었음)

- ① cmd_queue 확인 → 비었음
- ② rx_queue 확인 → 비었음
- ③ k_poll(K_FOREVER) → 스레드 blocked
 - └ CPU는 idle/sleep으로 진입 가능 (RX 인터럽트 또는 타이머가 큐에 넣을 때까지)

5. 변경 전/후 비교

항목	현재 (폴링)	최종 (2큐 + k_poll)
스레드 대기	<code>lora_recv()</code> 500ms 타임아웃	<code>k_poll(K_FOREVER)</code> 2큐 동시 대기
깨어나는 이유	타임아웃 만료 (주기적)	큐에 메시지 도착 시만

항목	현재 (폴링)	최종 (2큐 + k_poll)
RX 경로	스레드가 직접 recv 호출	ISR 콜백 → rx_queue → 스레드
TX 경로	별도 TX 큐 + K_NO_WAIT 폴링	cmd_queue에 TX_REQUEST
ACK 타임아웃	uptime 비교 (루프마다)	k_timer → cmd_queue에 ACK_TIMEOUT
RX 폭주 시 TX	같은 루프에서 경합	cmd_queue 우선 → TX 보호
RX 큐 포화 시 TX	TX도 같은 큐면 실패 가능	별도 큐라 영향 없음
Receive timeout 로그	500ms마다 출력	사라짐
유휴 시 CPU	주기적 깨움 (sleep 불가)	blocked → sleep 가능

6. 주의 사항

항목	설명
ISR 제약	lora_recv_async() 콜백은 ISR 컨텍스트 — mutex 획득, 긴 처리 불가. k_msgq_put(K_NO_WAIT)만 허용
반이중	SX1262는 TX/RX 동시 불가 — TX 전 lora_recv_async(NULL) 필수, TX 후 재등록 필수
RX 재등록	TX 완료 후 반드시 lora_recv_async(cb) 호출하여 RX 재개. 누락 시 수신 불가
in-flight	단일 in-flight 규칙은 lora_module_enqueue_tx() 또는 워커의 TX_REQUEST 처리에서 유지
k_poll 이벤트 초기화	k_poll_event는 매 루프마다 재초기화 필요 (Zephyr k_poll 규칙)

7. 관련 소스

파일	위치	역할
lora.h	zephyr/include/zephyr/drivers/lora.h:293	lora_recv_async() API 정의
sx12xx_common.c	zephyr/drivers/lora/sx12xx_common.c:310	sx12xx_lora_recv_async() 구현
lora_module.c	apps/system/src/lora_module.c:241	현재 서비스 루프 (변경 대상)

작성: Claude Code